Analyze A/B Test Results 🚀

This project will assure you have mastered the subjects covered in the statistics lessons. We have organized the current notebook into the following sections:

Specific programming tasks are marked with a **ToDo** tag.

Introduction 👋

A/B tests are very commonly performed by data analysts and data scientists. For this project, you will be working to understand the results of an A/B test run by an e-commerce website. Your goal is to work through this notebook to help the company understand if they should:

- Implement the new webpage,
- Keep the old webpage, or
- Perhaps run the experiment longer to make their decision.

Each **ToDo** task below has an associated quiz present in the classroom. Though the classroom quizzes are **not necessary** to complete the project, they help ensure you are on the right track as you work through the project, and you can feel more confident in your final submission meeting the rubric specification.

> **Tip**: Though it's not a mandate, students can attempt the classroom quizzes to ensure statistical numeric values are calculated correctly in many cases.

Part I - Probability 👀

To get started, let's import our libraries.

In [1]:

```
import pandas as pd
import numpy as np
import random
import matplotlib.pyplot as plt

from typing import List

%matplotlib inline
# We are setting the seed to assure
# you get the same answers on quizzes as we set up
random.seed(42)
```

ToDo 1.1

Now, read in the `ab_data.csv` data. Store it in `df`. Below is the description of the data, there are a total of 5 columns:

Data columns	Purpose	Valid values
user_id	Unique ID	Int64 values
timestamp	Time stamp when the user visited the webpage	-
group	In the current A/B experiment, the users are categorized into two broad groups. The `control` group users are expected to be served with `old_page`; and `treatment` group users are matched with the `new_page`. However, **some inaccurate rows** are present in the initial data, such as a `control` group user is matched with a `new_page`.	`['control', 'treatment']`
landing_page	It denotes whether the user visited the old or new webpage.	`['old_page', 'new_page']`
converted	It denotes whether the user decided to pay for the company's product. Here, `1` means yes, the user bought the product.	`[0, 1]`

 Use your dataframe to answer the questions in Quiz 1 of the classroom.

> **Tip**: Please save your work regularly.

a. Read in the dataset from the `ab_data.csv` file and take a look at the top few rows here:

In [2]:
```
data = pd.read_csv("datasets/ab_data.csv")
```

We can see the first `n` rows by using pandas' dataframe's `head` method

In [3]:
```
data.head(4) # it shows the first 4 rows
```

Out[3]:

	user_id	timestamp	group	landing_page	converted
0	851104	2017-01-21 22:11:48.556739	control	old_page	0
1	804228	2017-01-12 08:01:45.159739	control	old_page	0
2	661590	2017-01-11 16:55:06.154213	treatment	new_page	0
3	853541	2017-01-08 18:28:03.143765	treatment	new_page	0

b. Use the cell below to find the number of rows in the dataset.

We should use `shape` method which returns a tuple representing the dimensionality of our dataframe

In [4]:
```
data.shape
```

Out[4]:
```
(294478, 5)
```

We can also use df's `info` method for this:

c. The number of unique users in the dataset.

- https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.nunique.html?highlight=nunique#pandas.DataFrame.nunique

In [5]:
```
data.user_id.nunique()
```

Out[5]:
```
290584
```

d. The proportion of users converted.

In [6]:
```
data.converted.mean()
```

Out[6]:
```
0.11965919355605512
```

e. The number of times when the "group" is `treatment` but "landing_page" is not a `new_page`.

In [7]:
```
def calculate_number_of_times(
    df: pd.DataFrame,
    groups: List[str],
    pages: List[str]
) -> pd.Series:

    output: List[pd.DataFrame] = []

    index: int

    for index, group in enumerate(groups):
        output.append(
            df[
                df.group == group
            ][
                df.landing_page == pages[index]
            ].count()
        )

    return sum(output)


calculate_number_of_times(
    data,
    ["treatment", "control"],
    ["old_page", "new_page"]
)
```

Out[7]:
```
C:\Users\User\AppData\Local\Temp/ipykernel_5308/2375580965.py:13: UserWarning: Boolean Series key will be reindexed to match DataFrame index.
  df[
user_id         3893
timestamp       3893
group           3893
landing_page    3893
converted       3893
dtype: int64
```

f. Do any of the rows have missing values?

`isnull` - detects missing values on DF.

In [8]:
```
# You can see that there are no missing values on the dataset
data.isnull().sum()
```

Out[8]:
```
user_id         0
timestamp       0
group           0
landing_page    0
converted       0
dtype: int64
```

ToDo 1.2

In a particular row, the **group** and **landing_page** columns should have either of the following acceptable values:

user_id	timestamp	group	landing_page	converted
XXXX	XXXX	`control`	`old_page`	X
XXXX	XXXX	`treatment`	`new_page`	X

It means, the `control` group users should match with `old_page`; and `treatment` group users should matched with the `new_page`.

However, for the rows where `treatment` does not match with `new_page` or `control` does not match with `old_page`, we cannot be sure if such rows truly received the new or old wepage.

Use **Quiz 2** in the classroom to figure out how should we handle the rows where the group and landing_page columns don't match?

a. Now use the answer to the quiz to create a new dataset that meets the specifications from the quiz. Store your new dataframe in **df2**.

In [9]:
```
# Remove the inaccurate rows, and store the result in a new dataframe df

def create_dataframe(
    df: pd.DataFrame,
    groups: List[str],
    pages: List[str]
) -> pd.DataFrame:

    data_frames: List[pd.DataFrame] = [
        df[
            (df.group == group)
            &
            (df.landing_page == pages[index])
        ]
        for index, group in enumerate(groups)
    ]

    return pd.concat(data_frames)


df2 = create_dataframe(
        data,
```

```
        ["treatment", "control"],
        ["new_page", "old_page"]
)
```

In [10]:
```
# Double Check all of the incorrect rows were removed from df2 -
# Output of the statement below should be 0
df2[
    (
        (df2['group'] == 'treatment')
        ==
        (df2['landing_page'] == 'new_page')
    ) == False
].shape[0]
```

Out[10]:
```
0
```

ToDo 1.3

Use **df2** and the cells below to answer questions for **Quiz 3** in the classroom.

a. How many unique **user_id**s are in **df2**?

In [11]:
```
df2.user_id.nunique()
```

Out[11]:
```
290584
```

b. There is one **user_id** repeated in **df2**. What is it?

- Reference: https://thispointer.com/pandas-find-duplicate-rows-in-a-dataframe-based-on-all-or-selected-columns-using-dataframe-duplicated-in-python

In [12]:
```
df2[df2.duplicated(subset = 'user_id')]['user_id']
```

Out[12]:
```
2893    773192
Name: user_id, dtype: int64
```

The duplicated user's id is `773192`

c. Display the rows for the duplicate **user_id**?

In [13]:
```
df2[df2.duplicated(subset = 'user_id')]
```

Out[13]:

	user_id	timestamp	group	landing_page	converted
2893	773192	2017-01-14 02:55:59.590927	treatment	new_page	0

d. Remove **one** of the rows with a duplicate **user_id**, from the **df2** dataframe.

In [14]:
```
# Remove one of the rows with a duplicate user_id.
# Hint: The dataframe.drop_duplicates()
# may not work in this case because the rows with
# duplicate user_id are not entirely identical.
# Check again if the row with a duplicate user_id is deleted or not

df2.drop_duplicates(subset = 'user_id',inplace=True)
```

Checking again:

In [15]:
```
df2[df2.duplicated(subset = 'user_id')]
```

Out[15]:

user_id	timestamp	group	landing_page	converted

ToDo 1.4

Use **df2** in the cells below to answer the quiz questions related to **Quiz 4** in the classroom.

a. What is the probability of an individual converting regardless of the page they receive?

> **Tip**: The probability you'll compute represents the overall "converted" success rate in the population and you may call it $p_{population}$.

In [16]:
```
df2.converted.mean()
```

Out[16]:
```
0.11959708724499628
```

b. Given that an individual was in the `control` group, what is the probability they converted?

In [17]:
```
# We can create simple function for calculating this
calculate_probability = lambda group_name: df2[df2.group == group_name][
```

In [18]:
```
control_probability: np.float64 = calculate_probability("control")
control_probability
```

Out[18]:
```
0.1203863045004612
```

c. Given that an individual was in the `treatment` group, what is the probability they converted?

In [19]:
```
treatment_probability: np.float64 = calculate_probability("treatment")
treatment_probability
```

Out[19]:
```
0.11880806551510564
```

> **Tip**: The probabilities you've computed in the points (b). and (c). above can also be treated as conversion rate. Calculate the actual difference (`obs_diff`) between the conversion rates for the two groups. You will need that later.

In [20]:
```
# Calculate the actual difference (obs_diff)
# between the conversion rates for the two groups.

obs_diff: np.float64 = treatment_probability - control_probability
obs_diff
```

Out[20]:
```
-0.0015782389853555567
```

d. What is the probability that an individual received the new page?

In [21]:
```
(df2.landing_page == "new_page").mean()
```

Out[21]:
```
0.5000619442226688
```

e. Consider your results from parts (a) through (d) above, and explain below whether the new `treatment` group users lead to more conversions.

✅ **Answer**

1) The probability for individuals receiving new pages is equal to roughly 0.5 2) According to `obs_diff`, we can say that the number of people who are converted in the control group is slightly higher than people converted in the treatment group. 3) The percentage of people converted in the:

```
* **Control** group is `12.039%`
* **Treatment** group is `11.881%`
```

In [22]:
```
to_percentage = lambda probability: round(probability * 100, 3)

(
    "Treatment: " +
    f"{to_percentage(treatment_probability)} % ",
    "Control: " +
    f"{to_percentage(control_probability)} %"
)
```

Out[22]:
```
('Treatment: 11.881 % ', 'Control: 12.039 %')
```

🔰 Part II - A/B Test

Since a timestamp is associated with each event, you could run a hypothesis test continuously as long as you observe the events.

However, then the hard questions would be:

- Do you stop as soon as one page is considered significantly better than another or does it need to happen consistently for a certain amount of time?
- How long do you run to render a decision that neither page is better than another?

These questions are the difficult parts associated with A/B tests in general.

ToDo 2.1

For now, consider you need to make the decision just based on all the data provided.

> Recall that you just calculated that the "converted" probability (or rate) for the old page is *slightly* higher than that of the new page (ToDo 1.4.c).

If you want to assume that the old page is better unless the new page proves to be definitely better at a Type I error rate of 5%, what should be your null and alternative hypotheses (**H_0** and **H_1**)?

You can state your hypothesis in terms of words or in terms of **p_{old}** and **p_{new}**, which are the "converted" probability (or rate) for the old and new pages respectively.

- **Null Hypothesis: p_{new} - p_{old}** <= 0
- **Alternative Hypothesis: p_{new} - p_{old}** > 0

ToDo 2.2 - Null Hypothesis H_0 Testing

Under the null hypothesis H_0, assume that p_{new} and p_{old} are equal. Furthermore, assume that p_{new} and p_{old} both are equal to the **converted** success rate in the `df2` data regardless of the page. So, our assumption is:

p_{new} = p_{old} = $p_{population}$

In this section, you will:

- Simulate (bootstrap) sample data set for both groups, and compute the "converted" probability p for those samples.
- Use a sample size for each group equal to the ones in the `df2` data.
- Compute the difference in the "converted" probability for the two samples above.
- Perform the sampling distribution for the "difference in the converted probability" between the two simulated-samples over 10,000 iterations; and calculate an estimate.

Use the cells below to provide the necessary parts of this simulation. You can use **Quiz 5** in the classroom to make sure you are on the right track.

a. What is the **conversion rate** for p_{new} under the null hypothesis?

In [23]:
```
pNew = df2.converted.mean()
```

b. What is the **conversion rate** for p_{old} under the null hypothesis?

In [24]:
```
pOld = df2.converted.mean()
```

c. What is n_{new}, the number of individuals in the treatment group?

Hint: The treatment group users are shown the new page.

In [25]:
```
def calculate_count(page:str):
    return df2[df2.landing_page == page]["converted"].count()
```

In [26]:
```
n_new = calculate_count("new_page")
n_new
```

Out[26]:
```
145310
```

d. What is n_{old}, the number of individuals in the control group?

In [27]:
```
n_old=  calculate_count("old_page")
n_old
```

Out[27]:
```
145274
```

e. Simulate Sample for the `treatment` Group
Simulate n_{new} transactions with a conversion rate of p_{new} under the null hypothesis.

Hint: Use `numpy.random.choice()` method to randomly generate n_{new} number of values.
Store these n_{new} 1's and 0's in the `new_page_converted` numpy array.

In [28]:
```
# Simulate a Sample for the treatment Group
# I do not want to use magic variables 🙄

TIMES: int = 10_000


# This is a new solution and I want to
# show how it is faster ( it will be in the next blocks )

def simulate(new: bool = False, old: bool = False) -> np.ndarray:
    """
    According to feedback given to me from Udacity mentor
    I used np.random.binomial for simulating 🚀
    Thanks for it 😀
    I see it is `x70 000` faster than the oldest solution which I wrote
    - Old solution  ~ 70 seconds
    - This solution  ~ 0.002 second
    """

    return np.random.binomial(n_old, pOld, TIMES) if old \
        else np.random.binomial(n_new, pNew, TIMES)
```

If you want to see how it is fast, the next block may useful for you!

In [29]:
```
# My old solution 🤦
def simulate_old(new: bool = False, old: bool = False):
    size, p = None, None

    if new:
        size = n_new
        p = [pNew, (1-pNew)]

    else:
        size = n_old
        p = [pOld, (1-pOld)]

    return np.random.choice(
        [1, 0], size=size, p=p
    ).mean()
```

In [30]:
```
from time import time
from typing import Callable


def measure_time(function: Callable) -> str:
    # Simple function for measuring time execution
    start = time()

    function()

    end = time()

    return end - start


def old_solution():
    p_diffs = []

    TIMES: int = 10000

    for _ in range(TIMES):
        p_diffs.append(
            simulate_old(new=True) - simulate_old(old=True)
        )

    p_diffs = np.array(p_diffs, dtype=np.float64)

    p_diffs


def new_solution():
    p_diffs = simulate(new=True) - simulate(old=True)
    p_diffs


new, old = measure_time(new_solution), measure_time(old_solution)

"✨ x{} faster 🚀".format(old/new)
```

Out[30]:
```
'✨ x69142.40979571745 faster 🚀'
```

In [31]:
```
new_page_converted = simulate(new=True)
new_page_converted
```

Out[31]:
```
array([17373, 17618, 17184, ..., 17405, 17237, 17450])
```

f. Simulate Sample for the `control` Group

Simulate n_{old} transactions with a conversion rate of p_{old} under the null hypothesis.

Store these n_{old} 1's and 0's in the `old_page_converted` numpy array.

In [32]:
```
# Simulate a Sample for the control Group
old_page_converted = simulate(old=True)
old_page_converted
```

Out[32]:
```
array([17374, 17392, 17375, ..., 17534, 17334, 17302])
```

g. Find the difference in the "converted" probability $(p{'}_{new}$ - $p{'}_{old})$ for your simulated samples from the parts (e) and (f) above.

In [33]:
```
p_diffs = new_page_converted - old_page_converted
p_diffs
```

Out[33]:
```
array([  -1,  226, -191, ..., -129,  -97,  148])
```

h. Sampling distribution

Re-create `new_page_converted` and `old_page_converted` and find the $(p{'}_{new}$ - $p{'}_{old})$ value 10,000 times using the same simulation process you used in parts (a) through (g) above.

Store all $(p{'}_{new}$ - $p{'}_{old})$ values in a NumPy array called `p_diffs`.

In [34]:
```
# We have calculated it above 😉
p_diffs
```

Out[34]:
```
array([  -1,  226, -191, ..., -129,  -97,  148])
```

i. Histogram

Plot a histogram of the **p_diffs**. Does this plot look like what you expected? Use the matching problem in the classroom to assure you fully understand what was computed here.

Also, use `plt.axvline()` method to mark the actual difference observed in the `df2` data (recall `obs_diff`), in the chart.

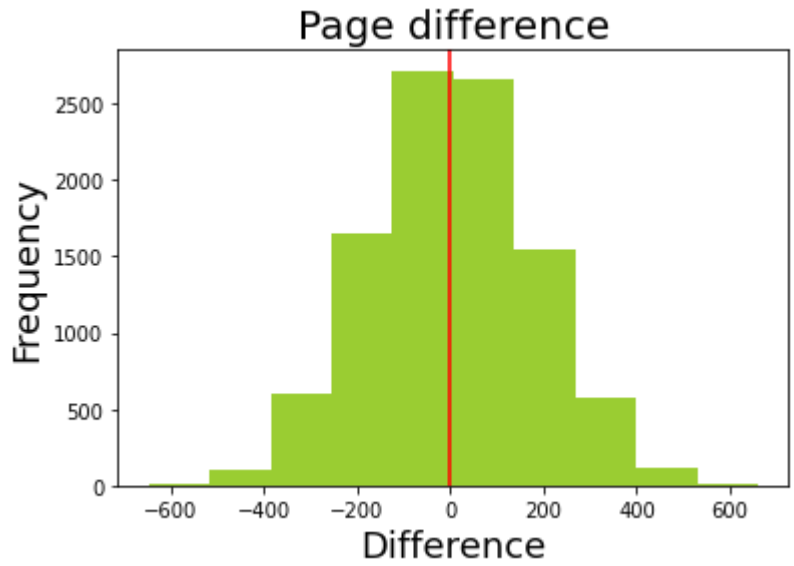
> **Tip**: Display title, x-label, and y-label in the chart.

In [50]:
```
plt.hist(p_diffs, color="yellowgreen")
plt.axvline(obs_diff, color="red")

plt.title("Page difference", fontsize=20)

plt.xlabel('Difference', fontsize=18)
plt.ylabel('Frequency', fontsize=18)
```

Out[50]:
```
Text(0, 0.5, 'Frequency')
```



j. What proportion of the **p_diffs** are greater than the actual difference observed in the `df2` data?

- `obs_diff` is defined above.
- `obs_diff` = treatment_probability - control_probability

In [36]:
```
(p_diffs > obs_diff).mean()
```

Out[36]:
```
0.5065
```

k. Please explain in words what you have just computed in part **j** above.

- What is this value called in scientific studies?
- What does this value signify in terms of whether or not there is a difference between the new and old pages? *Hint*: Compare the value above with the "Type I error rate (0.05)".

✅ **Answer**

1) In part `j`, we have computed the probability of our statistic + difference in means of converted old VS new pages 2) This value is called `p` value. 3) It means that the new page cannot convert more users than the old page.

I. Using Built-in Methods for Hypothesis Testing
We could also use a built-in to achieve similar results. Though using the built-in might be easier to code, the above portions are a walkthrough of the ideas that are critical to correctly thinking about statistical significance.

Fill in the statements below to calculate the:

- `convert_old`: number of conversions with the old_page
- `convert_new`: number of conversions with the new_page
- `n_old`: number of individuals who were shown the old_page
- `n_new`: number of individuals who were shown the new_page

In [37]:
```
import statsmodels.api as sm
```

```
def calculate_sum(group_name: str) -> int:
    return sum(df2[df2.group == group_name]["converted"])


# number of conversions with the old_page and new_page
convert_old, convert_new = calculate_sum("control"),\
    calculate_sum("treatment")

# number of individuals who were shown the old_page
# n_old - we have this

# number of individuals who received new_page
# n_new - we have this
```

m. Now use `sm.stats.proportions_ztest()` to compute your test statistic and p-value. Here is a helpful link on using the built in.

The syntax is:

```
proportions_ztest(count_array, nobs_array, alternative='larger')
```

where,

- `count_array` = represents the number of "converted" for each group
- `nobs_array` = represents the total number of observations (rows) in each group
- `alternative` = choose one of the values from `['two-sided', 'smaller', 'larger']` depending upon two-tailed, left-tailed, or right-tailed respectively.

> **Hint**:
> It's a two-tailed if you defined H_1 as $(p_{new} = p_{old})$.
> It's a left-tailed if you defined H_1 as $(p_{new} < p_{old})$.
> It's a right-tailed if you defined H_1 as $(p_{new} > p_{old})$.

The built-in function above will return the z_score, p_value.

About the two-sample z-test

Recall that you have plotted a distribution `p_diffs` representing the difference in the "converted" probability $(p{'}_{new}-p{'}_{old})$ for your two simulated samples 10,000 times.

Another way for comparing the mean of two independent and normal distribution is a **two-sample z-test**. You can perform the Z-test to calculate the Z_score, as shown in the equation below:

$$ Z_{score} = \frac{ (p{'}_{new}-p{'}_{old}) - (p_{new} - p_{old})}{ \sqrt{ \frac{\sigma^{2}_{new} }{n_{new}} + \frac{\sigma^{2}_{old} }{n_{old}} } } $$
where,

- $p{'}$ is the "converted" success rate in the sample
- p_{new} and p_{old} are the "converted" success rate for the two groups in the population.

- σ_{new} and σ_{new} are the standard deviation for the two groups in the population.
- n_{new} and n_{old} represent the size of the two groups or samples (it's same in our case)

> Z-test is performed when the sample size is large, and the population variance is known. The z-score represents the distance between the two "converted" success rates in terms of the standard error.

Next step is to make a decision to reject or fail to reject the null hypothesis based on comparing these two values:

- Z_{score}
- Z_{α} or $Z_{0.05}$, also known as critical value at 95% confidence interval. $Z_{0.05}$ is 1.645 for one-tailed tests, and 1.960 for two-tailed test. You can determine the Z_{α} from the z-table manually.

Decide if your hypothesis is either a two-tailed, left-tailed, or right-tailed test. Accordingly, reject OR fail to reject the null based on the comparison between Z_{score} and Z_{α}.

> Hint:
> For a right-tailed test, reject null if Z_{score} > Z_{α}.
> For a left-tailed test, reject null if Z_{score} < Z_{α}.

In other words, we determine whether or not the Z_{score} lies in the "rejection region" in the distribution. A "rejection region" is an interval where the null hypothesis is rejected iff the Z_{score} lies in that region.

Reference:

- Example 9.1.2 on this page/09%3A_Two-Sample_Problems/9.01%3A_Comparison_of_Two_Population_Means-_Large_Independent_Samples), courtesy www.stats.libretexts.org

---

> **Tip**: You don't have to dive deeper into z-test for this exercise. **Try having an overview of what does z-score signify in general.**

In [38]:
```
z_score, p_value = sm.stats.proportions_ztest(
    [convert_new, convert_old],
    [n_new, n_old],
    alternative='larger'
)

z_score, p_value
```

Out[38]:
```
(-1.3109241984234394, 0.9050583127590245)
```

n. What do the z-score and p-value you computed in the previous question mean for the conversion rates of the old and new pages? Do they agree with the findings in parts **j.** and

k.?

> **Tip**: Notice whether the p-value is similar to the one computed earlier. Accordingly, can you reject/fail to reject the null hypothesis? It is important to correctly interpret the test statistic and p-value.

✅ **Answer**

1) The score of Z is approximately `-1.3` which is less than the critical value of confidence (95%). Therefore we fail to reject Null Hypothesis.

In [39]:
```
from scipy.stats import norm

# Reference: https://www.youtube.com/watch?v=Yg9u3X2swWs

norm.cdf(z_score), norm.ppf(1 - (.05))
```

Out[39]:
```
(0.09494168724097551, 1.6448536269514722)
```

2) The results are the same as `J`. It means that we accept Null Hypothesis. 3) There is no doubt that old pages are better than new pages.

Part III - A regression approach

ToDo 3.1

In this final part, you will see that the result you achieved in the A/B test in Part II above can also be achieved by performing regression.

a. Since each row in the `df2` data is either a conversion or no conversion, what type of regression should you be performing in this case?

Logistic Regression, absolutely 😁

b. The goal is to use **statsmodels** library to fit the regression model you specified in part **a.** above to see if there is a significant difference in conversion based on the page-type a customer receives. However, you first need to create the following two columns in the `df2` dataframe:

1. `intercept` - It should be `1` in the entire column.
2. `ab_page` - It's a dummy variable column, having a value `1` when an individual receives the **treatment**, otherwise `0`.

In [40]:
```
# Add new column to dataframe (df2)
df2['intercept'] = 1

# pd.get_dummies - converts categorical
# variable into dummy/indicator variables.

df2[
    ['ab_page', 'old_page']
```

```
] = pd.get_dummies(df2['landing_page'])

df2.head()
```

Out[40]:

	user_id	timestamp	group	landing_page	converted	intercept	ab_page	old_pag
2	661590	2017-01-11 16:55:06.154213	treatment	new_page	0	1	1	
3	853541	2017-01-08 18:28:03.143765	treatment	new_page	0	1	1	
6	679687	2017-01-19 03:26:46.940749	treatment	new_page	1	1	1	
8	817355	2017-01-04 17:58:08.979471	treatment	new_page	1	1	1	
9	839785	2017-01-15 18:11:06.610965	treatment	new_page	1	1	1	

c. Use **statsmodels** to instantiate your regression model on the two columns you created in part (b). above, then fit the model to predict whether or not an individual converts.

In [41]:
```
model = sm.Logit(df2['converted'], df2[['intercept', 'ab_page']])
results = model.fit()
```

```
Optimization terminated successfully.
         Current function value: 0.366118
         Iterations 6
```

d. Provide the summary of your model below, and use it as necessary to answer the following questions.

In [42]:
```
results.summary()
```

Out[42]:

Logit Regression Results

Dep. Variable:	converted	**No. Observations:**	290584
Model:	Logit	**Df Residuals:**	290582
Method:	MLE	**Df Model:**	1
Date:	Sat, 30 Oct 2021	**Pseudo R-squ.:**	8.077e-06
Time:	11:55:30	**Log-Likelihood:**	-1.0639e+05
converged:	True	**LL-Null:**	-1.0639e+05
Covariance Type:	nonrobust	**LLR p-value:**	0.1899

	coef	std err	z	P>|z|	[0.025	0.975]
intercept	-1.9888	0.008	-246.669	0.000	-2.005	-1.973
ab_page	-0.0150	0.011	-1.311	0.190	-0.037	0.007

e. What is the p-value associated with **ab_page**? Why does it differ from the value you found in **Part II**?

Hints:

- What are the null and alternative hypotheses associated with your regression model, and how do they compare to the null and alternative hypotheses in **Part II**?
- You may comment on if these hypothesis (Part II vs. Part III) are one-sided or two-sided.
- You may also compare the current p-value with the Type I error rate (0.05).

✅ **Answer**

The `p` value which is found in the logistic regressions model is very different from than values that we found in the `Part II`. Because, in `Part II` we have used randomly chosen variables 😉 (10k iterations ...)

f. Now, you are considering other things that might influence whether or not an individual converts. Discuss why it is a good idea to consider other factors to add into your regression model. Are there any disadvantages to adding additional terms into your regression model?

✅ **Answer**

Taking into account other factors may lead to making better decisions. In my view, it can also make the hypotheses test results more accurate

Adding a lot of factors to the regression model may lead to unhelpful results. Particularly, if these factors do not have any impact ...

g. Adding countries
Now along with testing if the conversion rate changes for different pages, also add an effect based on which country a user lives in.

1. You will need to read in the **countries.csv** dataset and merge together your `df2` datasets on the appropriate rows. You call the resulting dataframe `df_merged`. Here are the docs for joining tables.

2. Does it appear that country had an impact on conversion? To answer this question, consider the three unique values, `['UK', 'US', 'CA']`, in the `country` column. Create dummy variables for these country columns.

   > **Hint:** Use `pandas.get_dummies()` to create dummy variables.
   > **You will utilize two columns for the three dummy variables.**

   Provide the statistical output as well as a written response to answer this question.

In [43]:
```
# Read the countries.csv
countries = pd.read_csv("datasets/countries.csv")
countries.head(4)
```

Out[43]:

	user_id	country
0	834778	UK
1	928468	US
2	822059	UK
3	711597	UK

In [44]:
```
# Join with the df2 dataframe
df = countries.set_index(
    "user_id"
).join(
    df2.set_index("user_id"),
    how='inner'
)
```

In [45]:
```
# We can check what countries are in DF
df.country.unique()
```

Out[45]:
```
array(['UK', 'US', 'CA'], dtype=object)
```

In [46]:
```
# Create the necessary dummy variables
df[['CA', 'US']] = pd.get_dummies(df['country'])[['CA', 'US']]
```

h. Fit your model and obtain the results

Though you have now looked at the individual factors of country and page on conversion, we would now like to look at an interaction between page and country to see if are there significant effects on conversion. **Create the necessary additional columns, and fit the new model.**

Provide the summary results (statistical output), and your conclusions (written response) based on the results.

> **Tip**: Conclusions should include both statistical reasoning, and practical reasoning for the situation.
>
> **Hints**:
>
> - Look at all of p-values in the summary, and compare against the Type I error rate (0.05).
> - Can you reject/fail to reject the null hypotheses (regression model)?
> - Comment on the effect of page and country to predict the conversion.

In [47]:
```
df['CA_new'] = df['CA'] * df['ab_page']
df['US_new'] = df['US'] * df['ab_page']

df.head(4)
```

Out[47]:

user_id	country	timestamp	group	landing_page	converted	intercept	ab_page	o
834778	UK	2017-01-14 23:08:43.304998	control	old_page	0	1	0	
928468	US	2017-01-23 14:44:16.387854	treatment	new_page	0	1	1	
822059	UK	2017-01-16 14:04:14.719771	treatment	new_page	1	1	1	
711597	UK	2017-01-22 03:14:24.763511	control	old_page	0	1	0	

In [48]:
```
sm.Logit(
    df['converted'],
    df[
        [
            'intercept', 'ab_page', 'CA',
            'US', 'CA_new', 'US_new'
        ]
    ]
).fit().summary()
```

```
Optimization terminated successfully.
         Current function value: 0.366109
         Iterations 6
```

Out[48]:

Logit Regression Results

Dep. Variable:	converted	**No. Observations:**	290584
Model:	Logit	**Df Residuals:**	290578
Method:	MLE	**Df Model:**	5
Date:	Sat, 30 Oct 2021	**Pseudo R-squ.:**	3.482e-05
Time:	11:55:32	**Log-Likelihood:**	-1.0639e+05
converged:	True	**LL-Null:**	-1.0639e+05
Covariance Type:	nonrobust	**LLR p-value:**	0.1920

	coef	std err	z	P>|z|	[0.025	0.975]
intercept	-1.9922	0.016	-123.457	0.000	-2.024	-1.961
ab_page	0.0108	0.023	0.475	0.635	-0.034	0.056
CA	-0.0118	0.040	-0.296	0.767	-0.090	0.066
US	0.0057	0.019	0.306	0.760	-0.031	0.043
CA_new	-0.0783	0.057	-1.378	0.168	-0.190	0.033
US_new	-0.0314	0.027	-1.181	0.238	-0.084	0.021

✅ Answer

In conclusion, I would say that there is no important `P value` and we will fail to reject the **null** 🧨

As you can see, we do have not enough evidence to suggest the new page is better than the old page.

Final Check 🔥

Congratulations! You have reached the end of the A/B Test Results project! You should be very proud of all you have accomplished!

> **Tip**: Once you are satisfied with your work here, check over your notebook to make sure that it satisfies all the specifications mentioned in the rubric. You should also probably remove all of the "Hints" and "Tips" like this one so that the presentation is as polished as possible.

Submission 🎈

You may either submit your notebook through the "SUBMIT PROJECT" button at the bottom of this workspace, or you may work from your local machine and submit on the last page of this project lesson.

1. Before you submit your project, you need to create a .html or .pdf version of this notebook in the workspace here. To do that, run the code cell below. If it worked correctly, you should get a return code of 0, and you should see the generated .html file in the workspace directory (click on the orange Jupyter icon in the upper left).

1. Alternatively, you can download this report as .html via the **File** > **Download as** submenu, and then manually upload it into the workspace directory by clicking on the orange Jupyter icon in the upper left, then using the Upload button.

1. Once you've done this, you can submit your project by clicking on the "Submit Project" button in the lower right here. This will create and submit a zip file with this .ipynb doc and the .html or .pdf version you created. Congratulations!

In [49]:
```
from subprocess import call
call(['python', '-m', 'nbconvert', 'Analyze_ab_test_results_notebook.ipy
```

Out[49]:
```
1
```